Design and Experimental Evaluation of a Cloud-Based IoT System for Electronic Toll Collection

Joaquín Barreyro
Escola Politécnica – University of São Paulo (USP)
São Paulo, Brazil
joaquinbarreyro@alumni.usp.br

Abstract*—Cloud computing provides scalable computational resources that can extend the capabilities of resource-constrained Internet of Things (IoT) devices. However, transferring sensing data to remote cloud services for processing introduces communication and processing delays that may affect applications requiring timely responses. This work presents the design, implementation, and experimental evaluation of a cloud-based IoT system for electronic toll collection. The prototype uses a Raspberry Pi equipped with a USB camera, a physical trigger representing vehicle detection, and actuators representing toll authorization. When a vehicle detection event is triggered, the Raspberry Pi captures an image and sends it to the cloud, where optical character recognition (OCR) is performed to identify the vehicle license plate. The detected plate is validated against a database of authorized vehicles, and the authorization decision is returned to the edge device. The cloud architecture is implemented using Amazon Web Services (AWS), integrating Amazon S3, AWS Lambda, Amazon Rekognition, DynamoDB, and AWS IoT Core. The experimental evaluation comprised 100 complete authorization cycles and resulted in a mean end-to-end latency of 4.61 s, with P95 and P99 values of 5.28 s and 5.68 s, respectively. Local image acquisition accounted for approximately 1.44 s, resulting in a mean post-capture latency of 3.17 s. Within the post-capture interval, image upload accounted for 46.2% of the mean latency, Lambda processing for 21.2%, and residual uninstrumented stages for 32.6%. The results show that measured cloud processing was not the dominant latency component and demonstrate the functional operation of the proposed cloud-centric architecture, while providing an initial indication of its temporal feasibility at prototype level and identifying image acquisition, data transfer, and asynchronous orchestration as the main opportunities for further optimization.

Keywords—Internet of Things, Cloud Computing, Electronic Toll Collection, Intelligent Transportation Systems, End-to-End Latency, Serverless Computing, Optical Character Recognition.

I. Introduction

Road transportation systems play a fundamental role in modern societies, supporting the mobility of people and goods while requiring increasingly efficient monitoring and management solutions **[1]**. In this context, Intelligent Transportation Systems (ITS) have become essential for improving road safety, traffic management, and electronic toll collection **[2]**.

Recent advances in computer vision and deep learning have substantially improved the capabilities of vehicle detection and classification systems **[3]**. In electronic toll collection applications, automatic license plate recognition (ALPR) provides a convenient mechanism for identifying vehicles and can be integrated with tolling and vehicle authorization systems **[4]**. However, image processing and optical character recognition (OCR) may require computational resources that are not always available in resource-constrained edge or Internet of Things (IoT) devices.

Edge Computing addresses this problem by placing computational resources close to the data source, reducing communication latency and dependence on remote infrastructure **[5]**. Conversely, Cloud Computing provides scalable computational and storage resources and enables centralized execution of computationally intensive applications **[6]**. This characteristic is particularly relevant when an infrastructure may comprise multiple edge devices, such as several lanes within a toll plaza and multiple toll plazas within the same highway concession. Cloud-based architectures can centralize vehicle information and authorization data, allow multiple edge devices to share common processing services, and reduce the computational and storage requirements of individual edge devices. Nevertheless, transferring images from edge devices to remote cloud infrastructure introduces communication and processing delays that may directly affect application response time **[7]**.

This trade-off is particularly relevant in toll gate applications. After a vehicle is detected, its identification, authorization, and the corresponding gate command must be completed within an acceptable time to avoid unnecessary vehicle waiting and degradation of the tolling operation. Therefore, the feasibility of moving vehicle identification and authorization to the cloud depends not only on the computational capabilities provided by cloud services, but also on the end-to-end latency introduced by image acquisition, network communication, remote processing, database access, and result transmission.

This work presents the design, implementation, and experimental evaluation of a cloud-based IoT system for electronic toll collection. A Raspberry Pi equipped with a USB camera is used as the edge IoT device. A physical trigger represents vehicle detection and initiates image acquisition. The captured image is transmitted to an Amazon Web Services (AWS) cloud infrastructure, where OCR is performed to extract the vehicle license plate. The identified plate is validated against a database of authorized vehicles, and the resulting decision is returned to the edge device. When authorization is granted, a physical output is activated to represent the opening of a toll gate.

The main objective of the experimental evaluation is to characterize the **end-to-end latency between the vehicle detection event and local actuator response**. Unlike analytical approaches based on estimated communication and processing times, the proposed prototype measures this latency using a real IoT device, physical sensor and actuators,

network communication, and deployed cloud services. In addition to the complete end-to-end response time, the experiment decomposes the latency into its main processing stages and evaluates the post-capture interval to distinguish local image acquisition from the communication and cloud-processing path. The obtained measurements are used to identify the dominant latency components and to discuss the feasibility and temporal implications of using cloud-based processing in electronic toll collection applications.

II. BACKGROUND AND RELATED WORK

This section presents the main concepts and related work that support the proposed architecture and experimental evaluation. Three aspects are considered: Automatic License Plate Recognition (ALPR) as the vehicle identification mechanism, Cloud and Edge Computing as alternative processing models for IoT applications, and end-to-end latency as a relevant performance metric for cloud-based systems interacting with physical devices.

A. Automatic License Plate Recognition

Automatic License Plate Recognition (ALPR), also referred to as Automatic Number Plate Recognition (ANPR), is a computer vision technology used to automatically identify vehicles from images or video streams. A typical ALPR process includes image acquisition, plate localization, character recognition using Optical Character Recognition (OCR), and conversion of the identified plate into machine-readable information **[4]**. ALPR has been applied to several transportation applications, including electronic toll collection and vehicle access control **[4]**. In this work, ALPR is used only as the vehicle identification mechanism, while the experimental evaluation focuses on the end-to-end latency of the complete cloud-based processing chain rather than on OCR accuracy.

B. Cloud and Edge Computing for IoT

Edge Computing places computational resources close to data sources, reducing communication latency and dependence on remote infrastructure **[5]**, **[7]**. In contrast, Cloud Computing provides scalable and centralized computational and storage resources that can support processing-intensive tasks and shared services **[6]**. For IoT applications, these approaches represent a trade-off between local processing and remote computational capacity. In the proposed system, the Raspberry Pi is intentionally used primarily for sensing, image acquisition, communication, and actuator control, while image recognition, authorization, and persistent data storage are provided by cloud services. This design reduces the computational and storage requirements of the edge device and provides a centralized infrastructure that can potentially be shared by multiple edge devices.

C. Latency in Cloud-Based IoT Systems

Latency is an important factor in cloud-based IoT applications, particularly when sensor data must be transmitted to remote infrastructure before an action can be performed. Communication delay depends on factors such as network conditions and the geographical distance between IoT devices and cloud data centers, while application processing introduces additional delays **[8]**. Therefore, evaluating only cloud processing time may not represent the response perceived by the physical system. In this work, latency is evaluated end-to-end, from the vehicle detection event at the edge device to the local actuator response, including image acquisition, data transmission, cloud processing, authorization, result transmission, and local actuation. In addition, the post-capture interval is separately analyzed to characterize the communication and cloud-processing path after an image becomes available at the edge device.

III. PROPOSED IoT ARCHITECTURE

The proposed architecture follows a cloud-centric approach in which the edge device is primarily responsible for sensing, image acquisition, communication, and local actuation, while computationally intensive image processing and persistent data storage are provided by managed cloud services. This section describes the physical components of the prototype and the event-driven cloud architecture used to perform vehicle identification and authorization.

A. System Overview

The proposed system implements a cloud-based IoT architecture for electronic toll collection. A Raspberry Pi 3 acts as the edge device and is connected to a USB camera, a physical trigger representing vehicle presence, and three LEDs representing the toll authorization state. Activation of the physical trigger initiates image acquisition. The captured image is transmitted to the AWS cloud infrastructure, where the license plate is extracted and compared with a database of authorized vehicles. The authorization result is then returned to the edge device. During processing, the LEDs provide a local indication of the system state, with the final output indicating whether the vehicle was authorized or rejected. In a real tolling installation, this authorization result could be used to control the corresponding toll gate actuator.

B. Cloud Architecture

The cloud infrastructure is implemented using AWS managed services. Captured images are uploaded to Amazon S3, which generates an event that triggers an AWS Lambda function responsible for coordinating the processing workflow. The function uses Amazon Rekognition to extract text from the image and normalizes the detected license plate before querying Amazon DynamoDB, which stores the authorized vehicle records. If no syntactically valid license plate is identified, the transaction is rejected without performing a DynamoDB query. The authorization result is then published through AWS IoT Core using MQTT and received by the Raspberry Pi. This event-driven architecture allows image processing, authorization, persistent storage, and device communication to be performed using managed cloud services.

Fig. 1 illustrates the complete processing flow of the proposed architecture, from vehicle detection and image acquisition at the edge device to cloud-based recognition and authorization, followed by the return of the authorization result through AWS IoT Core.

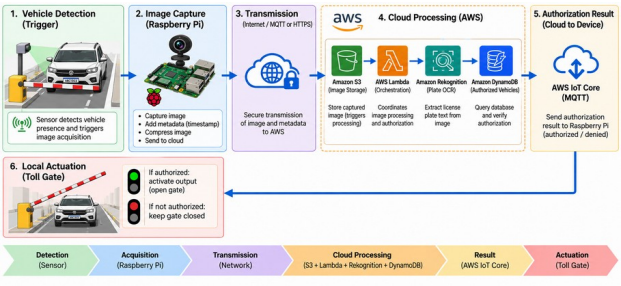


Fig. 1. Proposed cloud-based IoT architecture for electronic toll collection.

IV. SYSTEM IMPLEMENTATION

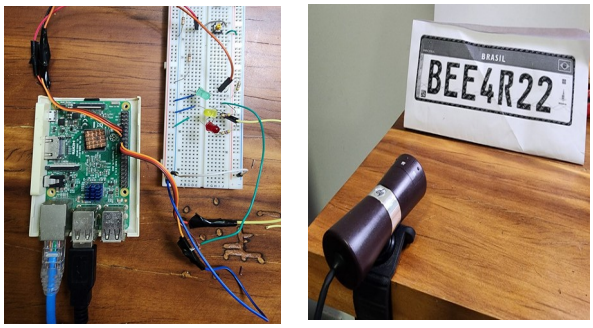
The prototype was implemented as a complete physical-to-cloud system, combining a resource-constrained edge device with managed AWS services. The implementation comprises the physical hardware and sensing components, the software responsible for image acquisition and device control, the cloud processing workflow, and the asynchronous communication mechanism used to return authorization decisions to the edge device.

A. Edge Device and Physical Prototype

The edge device is implemented using a Raspberry Pi 3 connected to a USB camera, a physical push button, and three LEDs. The push button emulates a vehicle-presence sensor and initiates the acquisition process. When the button is pressed, the Raspberry Pi records the initial timestamp and captures an image containing a printed vehicle license plate.

The three LEDs provide visual indication of the system state. The red LED represents the waiting or denied state, the yellow LED indicates that an authorization request is being processed, and the green LED indicates that the vehicle has been authorized. In a real tolling installation, the authorization output could be connected to the corresponding toll gate control system.

The physical prototype therefore reproduces the main interaction required for the experiment: a vehicle-presence event initiates image acquisition, the authorization process is performed remotely using cloud services, and the resulting decision produces a physical response at the edge device. Fig. 2 shows the physical prototype used during the experimental evaluation. Fig. 2(a) presents the Raspberry Pi 3 and the circuit implementing the vehicle-presence trigger and authorization indicators, while Fig. 2(b) shows the USB camera and one of the printed license plates used during image acquisition.



(a) (b)

Fig. 2. Physical prototype used in the experimental evaluation: (a) Raspberry Pi 3 edge device, vehicle-presence trigger, and LED-based authorization indicators; (b) USB camera and printed license plate used for image acquisition.

B. Edge Device Software

The edge device software was implemented in Python and is responsible for vehicle-presence detection, image acquisition, cloud communication, transaction management, and local actuator control. When the physical trigger is activated, the software creates a unique transaction identifier, records the initial timestamp, and initiates image acquisition using the USB camera.

Image acquisition is performed using `fswebcam`, which is invoked by the Python application for each transaction. The resulting JPEG image is associated with the transaction identifier and uploaded to Amazon S3. After the upload is completed, the application waits asynchronously for the corresponding authorization result through an MQTT subscription to AWS IoT Core.

The transaction identifier is used to correlate the received authorization message with the active vehicle event. A result is accepted only when its identifier corresponds to the current transaction, preventing a delayed response from a previous event from affecting a subsequent transaction. After a valid result is received, the software updates the LED indicators according to the authorization decision and records the final timestamp used for end-to-end latency measurement.

Fig. 3 summarizes the software execution flow implemented on the edge device, from vehicle detection and image acquisition to authorization result validation and local actuation.

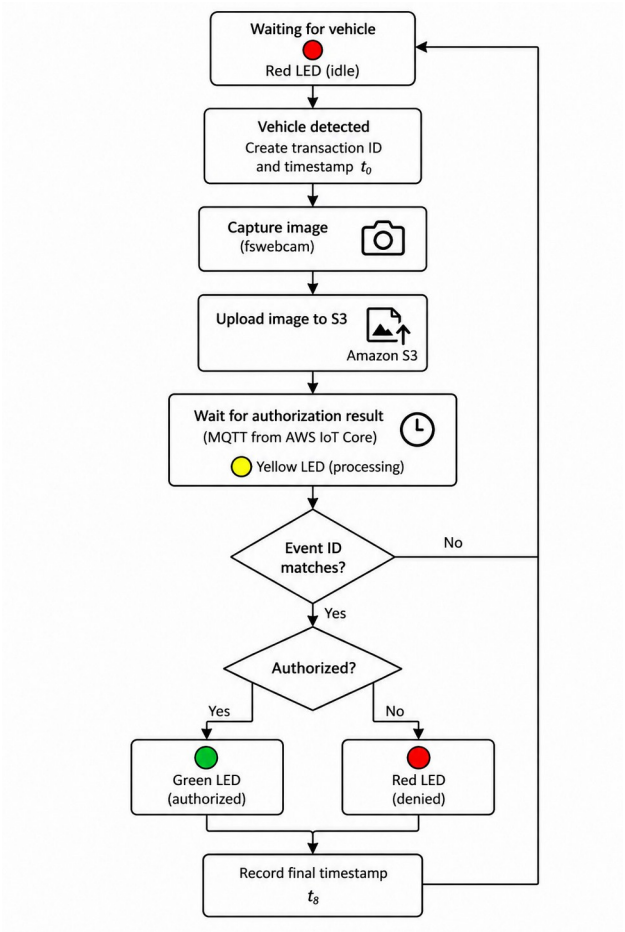


Fig. 3. Edge device software execution flow.

C. AWS Implementation

As illustrated in Fig. 1, the cloud processing workflow was implemented using Amazon S3, AWS Lambda, Amazon Rekognition, Amazon DynamoDB, and AWS IoT Core. Images uploaded by the edge device are stored in an Amazon S3 bucket. The creation of a new image object generates an event that automatically invokes an AWS Lambda function, initiating the cloud processing workflow.

The Lambda function requests text detection from Amazon Rekognition and processes the returned text to identify a candidate license plate. The detected text is normalized and validated according to the expected Brazilian license plate formats. If a syntactically valid plate is obtained, Amazon DynamoDB is queried to determine its authorization

status. If no valid plate identifier can be extracted, the transaction is rejected without performing a database query.

After the authorization decision is obtained, the Lambda function publishes a message containing the transaction identifier, detected license plate, and authorization status through AWS IoT Core using MQTT. The Raspberry Pi subscribes to the corresponding MQTT result topic and receives the authorization decision asynchronously.

This event-driven implementation allows image recognition, authorization, persistent data storage, and result distribution to be performed using managed cloud services, while the edge device remains responsible primarily for sensing, image acquisition, communication, and local actuation.

V. EXPERIMENTAL METHODOLOGY

A. End-to-End Latency Measurement

The experimental evaluation was designed to characterize the latency of the complete authorization process and to identify the contribution of its main processing stages. For each transaction, the initial timestamp was recorded on the Raspberry Pi when the physical vehicle detection trigger was activated. The final timestamp was recorded immediately after the authorization result was received and the corresponding LED output was activated. Since both timestamps were obtained using the same monotonic clock on the Raspberry Pi, the complete end-to-end latency was defined as

$$T_{E2E} = t_8 - t_0 \quad \textbf{(1)}$$

This measurement represents the response time observed by the physical prototype and includes local image acquisition, image upload, cloud processing, authorization, result transmission, and local actuation.

Intermediate processing times were also recorded. The Raspberry Pi measured the image acquisition time and the time required to upload the resulting image to Amazon S3. Within the AWS Lambda function, separate measurements were obtained for Amazon Rekognition and DynamoDB access, as well as for the total Lambda processing interval.

Image acquisition was implemented by invoking `fswebcam` for each transaction. Consequently, the measured acquisition time includes the overhead associated with starting the capture process and interacting with the USB camera. This local acquisition mechanism is implementation-dependent and can potentially be optimized, for example, by maintaining a persistent camera acquisition pipeline.

Since the primary objective of this work is to investigate the temporal impact of a cloud-centric architecture with minimal processing at the edge device, a second latency metric was defined by removing the measured local image acquisition time from the complete end-to-end latency:

$$T_{post} = T_{E2E} - T_{capture} \quad \textbf{(2)}$$

The complete end-to-end latency is reported to characterize the actual behavior of the implemented prototype, while the post-capture latency is used to analyze the communication and processing path after an image becomes available at the edge device.

B. Latency Decomposition

The post-capture latency was decomposed into three main components:

$$T_{post} = T_{upload} + T_{lambda} + T_{residual} \quad \textbf{(3)}$$

where T_{upload} represents the image transfer from the Raspberry Pi to Amazon S3, T_{lambda} represents the processing interval measured inside the Lambda function, and $T_{residual}$ represents the remaining uninstrumented latency.

The Lambda processing interval was further decomposed as:

$$T_{lambda} = T_{rekognition} + T_{dynamodb} + T_{lambdaOther} \quad \textbf{(4)}$$

where $T_{rekognition}$ is the time required for cloud-based text recognition, $T_{dynamodb}$ is the database query time, and $T_{lambdaOther}$ represents the remaining processing performed by the Lambda function.

The residual component includes delays that were not independently timestamped, such as S3 event propagation and Lambda invocation, transmission of the authorization result through AWS IoT Core using MQTT, and local software overhead. Therefore, this component is treated as residual latency and is not interpreted exclusively as network communication delay.

C. Experimental Procedure

A total of **100 authorization cycles** were performed using the complete physical and cloud infrastructure. Multiple printed license plates were presented to the USB camera, including plates registered as authorized and unauthorized in DynamoDB. Each cycle started with activation of the physical vehicle-presence trigger and continued through image acquisition, S3 upload, cloud-based text recognition, database verification, MQTT result transmission, and local actuator control.

If no valid license plate could be extracted from the recognized text, the transaction was rejected without querying DynamoDB. These executions were retained in the experimental dataset because they represent valid executions of the complete system rather than measurement failures.

For each transaction, the measured values were stored locally in CSV format. The resulting dataset was analyzed using minimum, maximum, arithmetic mean, median, sample standard deviation, and the 95th and 99th percentiles. No latency observations were removed from the dataset.

VI. RESULTS

A total of 100 authorization cycles were performed using the implemented prototype. The following analysis first presents the complete end-to-end latency observed by the physical system and then isolates the post-capture latency in order to characterize the communication and cloud-processing path. Finally, the post-capture interval is decomposed into its main measured components to identify their individual contributions to the overall response time.

A. End-to-End and Post-Capture Latency

Table I summarizes the latency measured by the physical prototype. The complete end-to-end latency, measured from activation of the vehicle detection trigger to local actuator response, presented a mean value of 4614.35 ms and a median of 4577.73 ms. The minimum and maximum observed

latencies were 3938.86 ms and 6055.89 ms, respectively. The 95th percentile was 5276.70 ms and the 99th percentile was 5677.25 ms.

The local image acquisition stage presented a mean latency of 1441.88 ms and low variability, with a standard deviation of 11.19 ms. After subtracting this acquisition interval, the resulting post-capture latency presented a mean of 3172.46 ms and a median of 3139.58 ms. The corresponding P95 and P99 values were 3838.91 ms and 4236.56 ms, respectively.

TABLE I. LATENCY RESULTS FOR 100 CYCLES

Metric	End-to-End (ms)	Capture (ms)	Post-Capture (ms)
Minimum	3938.86	1432.43	2503.39
Mean	**4614.35**	1441.88	**3172.46**
Median	4577.73	1438.07	3139.58
Std. deviation	368.18	11.19	367.03
P95	5276.70	1469.32	3838.91
P99	5677.25	1473.13	4236.56
Maximum	6055.89	1473.48	4616.66

Among the 100 measured transactions, 87 presented complete end-to-end latency below 5 s, while 99 completed below 6 s.

B. Post-Capture Latency Decomposition

Table II presents the decomposition of the post-capture latency into its main measured components. Image upload to Amazon S3 required an average of 1464.28 ms, while the processing interval measured inside AWS Lambda averaged 673.98 ms. The remaining residual latency averaged 1034.20 ms.

Relative to the mean post-capture latency of 3172.46 ms, image upload represented approximately 46.2%, Lambda processing 21.2%, and residual latency 32.6%. Table II also reports the P95 latency of each component, providing an indication of their behavior beyond the mean values.

TABLE II. POST-CAPTURE LATENCY DECOMPOSITION

Component	Mean (ms)	P95 (ms)	Contribution (%)
S3 upload	1464.28	1768.45	46.2%
Lambda processing	673.98	975.59	21.2%
Residual	1034.20	1447.46	32.6%
Post-capture total	3172.46	3838.91	100%

The residual component corresponds to the difference between the measured post-capture latency and the independently measured S3 upload and Lambda processing intervals. As defined in Section V, it includes uninstrumented stages of the asynchronous processing chain and is therefore not interpreted as network latency alone.

C. Cloud Processing Latency

Table III summarizes the latency of the individual components measured within the AWS Lambda processing stage, including Amazon Rekognition, DynamoDB access, and the remaining Lambda execution overhead. Amazon Rekognition was the dominant component, with a mean processing time of 652.74 ms and a median of 625.18 ms. Its P95 value was 898.97 ms.

Seven of the 100 transactions did not produce a valid plate identifier and therefore did not require a DynamoDB query. Considering the 93 transactions in which database access was performed, DynamoDB presented a mean latency of 22.05 ms and a median of 16.45 ms.

The remaining Lambda execution overhead was small compared with the external service calls, with a mean value of 0.73 ms.

TABLE III. CLOUD PROCESSING LATENCY

Component	Mean (ms)	Median (ms)	P95 (ms)	Max (ms)
Rekognition	**652.74**	625.18	898.97	1117.90
DynamoDB*	**22.05**	16.45	33.89	216.53
Lambda internal overhead	**0.73**	0.23	0.65	19.04
Lambda processing	**673.98**	636.24	975.59	1160.21

*DynamoDB statistics consider only the 93 transactions in which a database query was performed.

Overall, the measured Lambda processing interval presented a mean latency of 673.98 ms and a P95 of 975.59 ms, with Amazon Rekognition accounting for most of this processing time.

VII. DISCUSSION

The experimental results show that the implemented cloud-based toll authorization system achieved a mean end-to-end response time of 4.61 s. However, approximately 1.44 s of this interval was associated with local image acquisition, resulting in a mean post-capture latency of 3.17 s. The image acquisition stage exhibited very low variability, with a standard deviation of only 11.19 ms, compared with 368.18 ms for the complete end-to-end latency. This indicates that, although image acquisition introduced a significant and nearly constant delay, most of the observed latency variation originated in the processing stages after image capture.

The decomposition of the post-capture latency provides further insight into the behavior of the cloud-based architecture. Image upload to Amazon S3 represented the largest measured component, accounting for 46.2% of the mean post-capture latency. In comparison, the processing interval measured inside AWS Lambda represented 21.2%, while the residual component accounted for the remaining 32.6%. Therefore, the measured cloud processing time itself was not the dominant contributor to the response time. Instead, a substantial portion of the latency was associated with image transfer and with stages of the asynchronous processing chain that were not independently instrumented.

Within the Lambda processing interval, Amazon Rekognition was the dominant component, with a mean latency of 652.74 ms compared with 22.05 ms for DynamoDB access in transactions where a database query was performed. The remaining Lambda execution overhead was negligible, with a mean value of 0.73 ms. These results indicate that most of the measured Lambda processing time was associated with the external image recognition service rather than with the authorization logic or database access.

The residual latency requires a more cautious interpretation. Because timestamps were not collected at every intermediate point of the asynchronous processing chain, this component cannot be attributed exclusively to network communication. It includes, among other unmeasured stages, S3 event propagation and Lambda invocation, publication and delivery of the authorization result through AWS IoT Core, and local software processing before actuator response. Additional instrumentation would be required to isolate the contribution of each of these stages.

The local image acquisition stage represents another relevant implementation-specific factor. In the prototype, `fswebcam` was launched as a separate process for each transaction, and the measured acquisition interval therefore includes process startup and interaction with the USB camera. A persistent acquisition pipeline could potentially reduce this local overhead, although the magnitude of this improvement was not evaluated in the present work.

The proposed architecture was also motivated by the resource constraints typically associated with edge devices. A single toll plaza may contain multiple lanes and therefore multiple acquisition devices, while a highway concession may operate several toll plazas distributed over a large geographical area. Performing all storage and computational tasks locally would require sufficient processing and storage capacity at each individual edge device. In the proposed architecture, these requirements are shifted to managed cloud services: Amazon S3 and DynamoDB provide centralized storage and data persistence, while AWS Lambda and Amazon Rekognition provide on-demand processing capabilities. This approach allows the edge device to remain relatively simple and resource-constrained, while the centralized and scalable nature of the cloud architecture provides a common processing infrastructure that can potentially support deployments ranging from a small number of devices to multiple lanes and toll plazas. The scalability of such a deployment, however, was not experimentally evaluated in this work.

License plate recognition was selected for this proof of concept because it represents a direct and relevant image-processing task for electronic toll authorization. However, the same architectural approach is not limited to OCR. Additional images or sensor data could be processed by cloud services to perform other computationally intensive tolling tasks. One relevant example is image-based vehicle classification, in which visual characteristics of the vehicle could be analyzed to determine its toll category. Therefore, the cloud processing layer evaluated in this work can be viewed as a general computational resource for edge devices rather than as an architecture restricted to license plate recognition.

The measured response times can also be examined in the context of operational parameters adopted in Brazilian highway concessions. The two concession programs analyzed in this work establish maximum toll-plaza queue lengths of 200 m under normal operating conditions and 400 m during periods of increased demand [9], [10]. In addition, the concession program in [9] specifies that 95% of manual toll collection operations should be completed within 15 s. These contractual parameters are concession-specific and are used here only to provide an operational reference for the time scale of the authorization process evaluated in this work.

A simplified capacity analysis provides additional context. Assuming an average longitudinal occupancy of 8 m per vehicle, including vehicle length and inter-vehicle spacing, a 200-m queue would correspond to approximately 25 vehicles, while a 400-m queue would correspond to approximately 50 vehicles. Considering the waiting-time parameters specified in [9], 25 vehicles served within 5 min and 50 vehicles served within 10 min both correspond to an average service interval of approximately 12 s per vehicle, or 300 vehicles/h per lane. In comparison, the prototype achieved a mean end-to-end authorization latency of 4.61 s and a P95 latency of 5.28 s. If authorization latency were conservatively treated as a fully sequential service interval, these values would correspond to approximately 780 and 682 authorization cycles/h, respectively.

This simplified comparison should not be interpreted as a measurement of actual toll-lane capacity or as evidence of contractual compliance. The experiment measured authorization latency rather than complete vehicle service time. Actual lane capacity also depends on barrier movement, vehicle acceleration and departure, driver behavior, inter-vehicle spacing, traffic demand, and the possibility of concurrent transaction processing. Nevertheless, the comparison indicates that the measured authorization latency operates on a time scale compatible with the operational references considered here and provides an initial indication of the temporal feasibility of the proposed cloud-based architecture at prototype level.

Finally, the results identify several opportunities for further improvement. The local capture mechanism could be replaced by a persistent camera acquisition pipeline, while image size, compression, and upload strategies could be optimized to reduce the 1.46 s mean S3 transfer time. Additional instrumentation could also isolate the components currently grouped as residual latency. A direct comparison with edge-based license plate recognition would be particularly relevant, since local processing could eliminate image upload and part of the cloud orchestration path, but would introduce inference processing at the edge device. Such a comparison requires a separate experiment and cannot be inferred from the measurements obtained in this work.

VIII. CONCLUSION

This work presented the design, implementation, and experimental evaluation of a cloud-based IoT architecture for electronic toll authorization. The prototype integrated a Raspberry Pi edge device, physical vehicle detection, image acquisition, Amazon S3, AWS Lambda, Amazon Rekognition, DynamoDB, AWS IoT Core, and local actuator control, demonstrating a complete end-to-end interaction between physical devices and cloud services.

The experimental evaluation comprised 100 complete authorization cycles. The implemented system achieved a mean end-to-end latency of 4.61 s, with P95 and P99 values of 5.28 s and 5.68 s, respectively. Local image acquisition accounted for approximately 1.44 s, resulting in a mean post-capture latency of 3.17 s. The decomposition of this interval showed that image upload represented 46.2% of the mean post-capture latency, Lambda processing 21.2%, and residual uninstrumented stages 32.6%. Within the Lambda processing interval, Amazon Rekognition was the dominant component, while database access and internal authorization logic introduced comparatively small delays.

The results indicate that **measured cloud processing itself** was not the main source of latency in the evaluated architecture. Instead, image transfer, local image acquisition,

and uninstrumented stages of the asynchronous processing chain represented the largest opportunities for improvement. The comparison with operational parameters adopted in Brazilian highway concession programs also provides an initial indication that the measured authorization latency is compatible with the time scale required for toll-plaza operation [9], [10]. This comparison, however, should not be interpreted as evidence of contractual compliance or as a measurement of actual lane capacity, since these also depend on traffic demand, vehicle dynamics, barrier operation, and other operational factors.

Overall, the experiment **demonstrated the functional operation of the proposed cloud-centric architecture and provided an initial indication of its temporal feasibility at prototype level**. Future work should investigate persistent image acquisition, optimized image transfer, finer instrumentation of the asynchronous processing chain, and direct comparison with edge-based license plate recognition. Experiments under realistic traffic loads and concurrent vehicle processing would also be required to evaluate throughput, queue formation, and the suitability of the architecture for production deployment.

REFERENCES

[1] OECD, "Road Transport Infrastructure and Networks," OECD Transport Statistics, Paris, France, 2021.

[2] L. Zhu, F. R. Yu, Y. Wang, B. Ning, and T. Tang, "Big Data Analytics in Intelligent Transportation Systems: A Survey," IEEE Transactions on Intelligent Transportation Systems, vol. 20, no. 1, pp. 383–398, Jan. 2019.

[3] Z. Zhao, P. Zheng, S. Xu, and X. Wu, "Object Detection with Deep Learning: A Review," IEEE Transactions on Neural Networks and Learning Systems, vol. 30, no. 11, pp. 3212–3232, Nov. 2019.

[4] J. Tang, L. Wan, J. Schooling, P. Zhao, J. Chen, and S. Wei, "Automatic number plate recognition (ANPR) in smart cities: A systematic review on technological advancements and application cases," *Cities*, vol. 129, Art. no. 103833, Oct. 2022, doi: 10.1016/j.cities.2022.103833.

[5] M. Satyanarayanan, "The Emergence of Edge Computing," Computer, vol. 50, no. 1, pp. 30–39, Jan. 2017.

[6] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud Computing and Emerging IT Platforms: Vision, Hype, and Reality for Delivering Computing as the 5th Utility," Future Generation Computer Systems, vol. 25, no. 6, pp. 599–616, Jun. 2009.

[7] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," IEEE Internet of Things Journal, vol. 3, no. 5, pp. 637–646, Oct. 2016.

[8] K. Zen, S. Mohanan, S. Tarmizi, N. Annuar, and N. U. Sama, "Latency Analysis of Cloud Infrastructure for Time-Critical IoT Use Cases," in *Proc. 2022 Applied Informatics International Conference (AiIC)*, 2022, pp. 111–116, doi: 10.1109/AiIC54368.2022.9914601.

[9] Agência Nacional de Transportes Terrestres (ANTT), *Programa de Exploração Rodoviária – PER: BR-101/ES/BA, Trecho Divisa ES/RJ – Entr. BA-698 (Acesso a Mucuri), Edital de Concessão nº 001/2011*, updated through the 5th Ordinary Revision and 6th Extraordinary Revision, Brasília, Brazil, 2018.

[10] Agência Nacional de Transportes Terrestres (ANTT), *Contrato de Concessão nº 4/2025 – Rota Sertaneja: Programa de Exploração da Rodovia – PER*, Brasília, Brazil, 2025